Introduction

↑ Back to top

What is Symbolic Computation?

Symbolic computation deals with the computation of mathematical objects symbolically. This means that the mathematical objects are represented exactly, not approximately, and mathematical expressions with unevaluated variables are left in symbolic form.

Let's take an example. Say we wanted to use the built-in Python functions to compute square roots. We might do something like this

```
>>> import math
>>> math.sqrt(9)
3.0
```

9 is a perfect square, so we got the exact answer, 3. But suppose we computed the square root of a number that isn't a perfect square

```
>>> math.sqrt(8)
2.82842712475
```

Here we got an approximate result. 2.82842712475 is not the exact square root of 8 (indeed, the actual square root of 8 cannot be represented by a finite decimal, since it is an irrational number). If all we cared about was the decimal form of the square root of 8, we would be done.

But suppose we want to go further. Recall that $\sqrt{8} = \sqrt{4 \cdot 2} = 2\sqrt{2}$. We would have a hard time deducing this from the above result. This is where symbolic computation comes in. With a symbolic computation system like SymPy, square roots of numbers that are not perfect squares are left unevaluated by default

```
>>> import sympy
>>> sympy.sqrt(3)
sqrt(3)
```

Furthermore—and this is where we start to see the real power of symbolic computation—symbolic results can be symbolically simplified.

```
>>> sympy.sqrt(8)
2*sqrt(2)
```

A More Interesting Example

The above example starts to show how we [illegible]ate irrational numbers exactly using SymPy. But it is much more powerful than [illegible]c computation systems (which by the way, are also often called computer algebra systems, or just CASs) such as SymPy are capable of computing symbolic expressions with variables.

↑ Back to top

As we will see later, in SymPy, variables are defined using `symbols`. Unlike many symbolic manipulation systems, variables in SymPy must be defined before they are used (the reason for this will be discussed in the next section).

Let us define a symbolic expression, representing the mathematical expression $x + 2y$.

```
>>> from sympy import symbols
>>> x, y = symbols('x y')
>>> expr = x + 2*y
>>> expr
x + 2*y
```

Note that we wrote `x + 2*y` just as we would if `x` and `y` were ordinary Python variables. But in this case, instead of evaluating to something, the expression remains as just `x + 2*y`. Now let us play around with it:

```
>>> expr + 1
x + 2*y + 1
>>> expr - x
2*y
```

Notice something in the above example. When we typed `expr - x`, we did not get `x + 2*y - x`, but rather just `2*y`. The `x` and the `-x` automatically canceled one another. This is similar to how `sqrt(8)` automatically turned into `2*sqrt(2)` above. This isn't always the case in SymPy, however:

```
>>> x*expr
x*(x + 2*y)
```

Here, we might have expected $x(x + 2y)$ to transform into $x^2 + 2xy$, but instead we see that the expression was left alone. This is a common theme in SymPy. Aside from obvious simplifications like $x - x = 0$ and $\sqrt{8} = 2\sqrt{2}$, most simplifications are not performed automatically. This is because we might prefer the factored form $x(x + 2y)$, or we might prefer the expanded form $x^2 + 2xy$. Both forms are useful in different circumstances. In SymPy, there are functions to go from one form to the other

```
>>> from sympy import expand, factor
>>> expanded_expr = expand(x*expr)
>>> expanded_expr
x**2 + 2*x*y
>>> factor(expanded_expr)
x*(x + 2*y)
```

↑ Back to top

The Power of Symbolic Computation

The real power of a symbolic computation system such as SymPy is the ability to do all sorts of computations symbolically. SymPy can simplify expressions, compute derivatives, integrals, and limits, solve equations, work with matrices, and much, much more, and do it all symbolically. It includes modules for plotting, printing (like 2D pretty printed output of math formulas, or $\LaTeX$), code generation, physics, statistics, combinatorics, number theory, geometry, logic, and more. Here is a small sampling of the sort of symbolic power SymPy is capable of, to whet your appetite.

```
>>> from sympy import *
>>> x, t, z, nu = symbols('x t z nu')
```

This will make all further examples pretty print with unicode characters.

```
>>> init_printing(use_unicode=True)
```

Take the derivative of $\sin(x)e^x$.

```
>>> diff(sin(x)*exp(x), x)
 x           x
ℯ ⋅sin(x) + ℯ ⋅cos(x)
```

Compute $\int (e^x \sin(x) + e^x \cos(x))\, dx$.

```
>>> integrate(exp(x)*sin(x) + exp(x)*cos(x), x)
 x
ℯ ⋅sin(x)
```

Compute $\int_{-\infty}^{\infty} \sin(x^2)\, dx$.

```
>>> integrate(sin(x**2), (x, -oo, oo))
√2⋅√π
─────
  2
```

Find $\lim_{x \to 0} \frac{\sin(x)}{x}$.

```
>>> limit(sin(x)/x, x, 0)
1
```

↑ Back to top

Solve $x^2 - 2 = 0$.

```
>>> solve(x**2 - 2, x)
[-√2, √2]
```

Solve the differential equation $y'' - y = e^t$.

```
>>> y = Function('y')
>>> dsolve(Eq(y(t).diff(t, t) - y(t), exp(t)), y(t))
            -t   ⎛     t⎞  t
y(t) = C₂⋅ℯ   + ⎜C₁ + ─⎟⋅ℯ
                 ⎝     2⎠
```

Find the eigenvalues of $\begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix}$.

```
>>> Matrix([[1, 2], [2, 2]]).eigenvals()
⎧3   √17     3   √17    ⎫
⎨─ - ───: 1, ─ + ───: 1⎬
⎩2    2      2    2     ⎭
```

Rewrite the Bessel function $J_\nu\left(z\right)$ in terms of the spherical Bessel function $j_\nu(z)$.

```
>>> besselj(nu, z).rewrite(jn)
√2⋅√z⋅jn(ν - 1/2, z)
────────────────────
         √π
```

Print $\int_0^\pi \cos^2\left(x\right) dx$ using $\LaTeX$.

```
>>> latex(Integral(cos(x)**2, (x, 0, pi)))
\int\limits_{0}^{\pi} \cos^{2}{\left(x \right)}\, dx
```

Why SymPy?

There are many computer algebra systems out there. This Wikipedia article lists many of them. What makes SymPy a better choice than the alternatives?

First off, SymPy is completely free. It is open source, and licensed under the liberal BSD license, so you can modify the source code and even sell it if you want to. This contrasts with popular commercial systems like Maple or Mathematica that cost hundreds of dollars in licenses.

Second, SymPy uses Python. Most computer algebra systems invent their own language. Not SymPy. SymPy is written entirely in Python, and is executed entirely in Python. This means that if you already know Python, it is much easier to get started with SymPy, because you already know the syntax (and if you don't know Python, it is really easy to learn). We already know that Python is a well-designed, battle-tested language. The SymPy developers are confident in their abilities in writing mathematical software, but programming language design is a completely different thing. By reusing an existing language, we are able to focus on those things that matter: the mathematics.

Back to top

Another computer algebra system, Sage also uses Python as its language. But Sage is large, with a download of over a gigabyte. An advantage of SymPy is that it is lightweight. In addition to being relatively small, it has no dependencies other than Python, so it can be used almost anywhere easily. Furthermore, the goals of Sage and the goals of SymPy are different. Sage aims to be a full featured system for mathematics, and aims to do so by compiling all the major open source mathematical systems together into one. When you call some function in Sage, such as `integrate`, it calls out to one of the open source packages that it includes. In fact, SymPy is included in Sage. SymPy on the other hand aims to be an independent system, with all the features implemented in SymPy itself.

A final important feature of SymPy is that it can be used as a library. Many computer algebra systems focus on being usable in interactive environments, but if you wish to automate or extend them, it is difficult to do. With SymPy, you can just as easily use it in an interactive Python environment or import it in your own Python application. SymPy also provides APIs to make it easy to extend it with your own custom functions.

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024